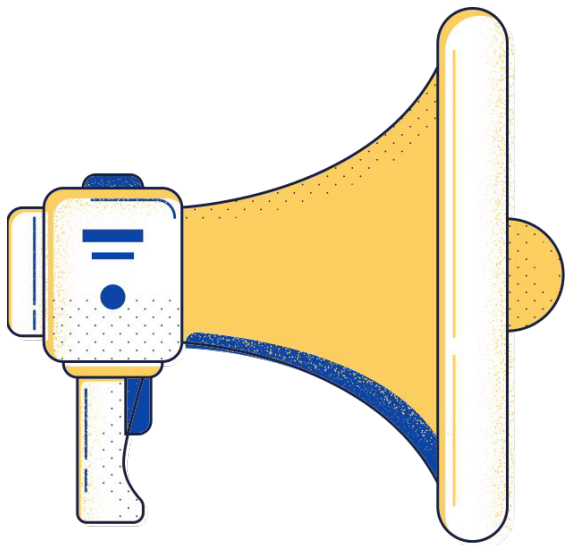


Sistema de Cadastro e Busca





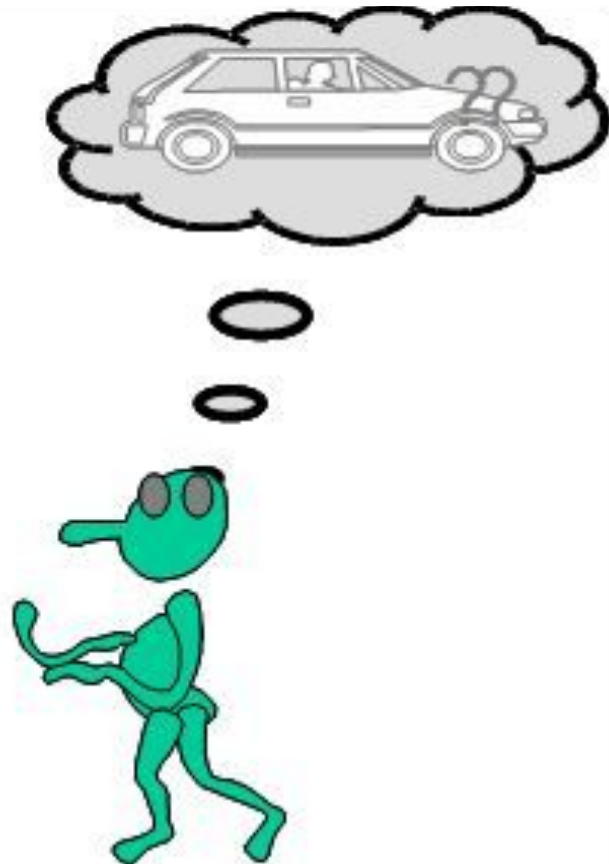
Objetivo

Implementar um sistema de gerenciamento de informações de usuários, utilizando estruturas de dados como array e objetos.

Lembram o que é um
requisito?



Cliente



Equipe do
Projeto

Gerenciamento de usuário

Requisitos?



Requisitos obrigatórios

- Cadastro de usuário
- Listagem de usuários
- Atualização de usuário
- Remoção de usuário

Requisitos obrigatórios

- Cadastro de usuário
 - Deve ser gerado um id sequencial para cada usuário
 - Deve ser adicionado o e-mail, nome e uma lista de telefones do usuário
 - Não permitir o cadastro de usuário com e-mail duplicado
 - Exibir mensagem de erro em caso de e-mail duplicado

Requisitos obrigatórios

- Busca de usuários
 - O sistema deve listar todos os usuários com suas informações
 - Telefones devem ser exibidos como uma lista associada

Requisitos obrigatórios

- Atualização de usuário
 - O usuário pode atualizar o nome, o e-mail ou adicionar ou remover um telefone associado
 - Ao atualizar o e-mail, deve garantir o novo não está em uso
 - Exibir mensagem de erro se o novo e-mail já estiver em uso

Requisitos obrigatórios

- Remoção de contato
 - Permitir a remoção de um contato especificando um ID
 - Solicitar mensagem de confirmação antes de proceder

**Lembram da
avaliação do
módulo 6?**

Menu de interação

```
1 // index.js
2 const prompt = require('prompt-sync')();
3 const listarContatos = require('./listarContatos');
4 const adicionarContato = require('./adicionarContato');
5 const atualizarContato = require('./atualizarContato');
6 const removerContato = require('./removerContato');
7
8 function mainMenu() {
9   console.log(`
10     1. Listar contatos
11     2. Adicionar contato
12     3. Atualizar contato
13     4. Remover contato
14     5. Sair
15   `);
16
17   const opcao = prompt('Escolha uma opção: ');
18
19   switch (opcao) {
20     case '1':
21       listarContatos();
22       mainMenu();
23     break;
```

```
24   case '2':
25     const nome = prompt('Nome: ');
26     const telefone = prompt('Telefone: ');
27     const email = prompt('Email: ');
28     adicionarContato({ id: Date.now(), nome, telefone, email });
29     console.log('Contato adicionado com sucesso!');
30     mainMenu();
31     break;
32   case '3':
33     const idAtualizar = parseInt(prompt('ID do contato a atualizar: '));
34     const novoNome = prompt('Novo nome: ');
35     const novoTelefone = prompt('Novo telefone: ');
36     const novoEmail = prompt('Novo email: ');
37     atualizarContato(idAtualizar, { nome: novoNome, telefone: novoTelefone, email: novoEmail });
38     console.log('Contato atualizado com sucesso!');
39     mainMenu();
40     break;
41   case '4':
42     const idRemover = parseInt(prompt('ID do contato a remover: '));
43     removerContato(idRemover);
44     console.log('Contato removido com sucesso!');
45     mainMenu();
46     break;
47   case '5':
48     break;
49   default:
50     console.log('Opção inválida!');
51     mainMenu();
52   }
53 }
54
55 mainMenu();
```


Cadastro de usuário



```
1 //adicionarContato.js
2
3 let contatos = require('./contatos');
4
5 function adicionarContato(contato) {
6   contatos.push(contato);
7 }
8
9 module.exports = adicionarContato;
```



```
1 //contatos.js
2
3 module.exports = [
4   { id: 1, nome: 'Alice', telefone: '1234-5678', email: 'alice@example.com' },
5   { id: 2, nome: 'Bob', telefone: '8765-4321', email: 'bob@example.com' },
6   { id: 3, nome: 'Carol', telefone: '5678-1234', email: 'carol@example.com' }
7 ];
```

Listagem de usuário



```
1 //listarContatos.js
2
3 let contatos = require('./contatos');
4
5 function listarContatos() {
6   contatos.forEach(contato => {
7     console.log(`ID: ${contato.id}, Nome: ${contato.nome}, Telefone: ${contato.telefone}, Email: ${contato.email}`);
8   });
9 }
10
11 module.exports = listarContatos;
```



```
1 //contatos.js
2
3 module.exports = [
4   { id: 1, nome: 'Alice', telefone: '1234-5678', email: 'alice@example.com' },
5   { id: 2, nome: 'Bob', telefone: '8765-4321', email: 'bob@example.com' },
6   { id: 3, nome: 'Carol', telefone: '5678-1234', email: 'carol@example.com' }
7 ];
```

Atualizar usuário



```
1 //atualizarContato.js
2
3 let contatos = require('./contatos');
4
5 function atualizarContato(id, novoContato) {
6     const index = contatos.findIndex(contato => contato.id === id);
7     if (index !== -1) {
8         contatos[index] = { id, ...novoContato };
9     }
10 }
11
12 module.exports = atualizarContato;
```



```
1 //contatos.js
2
3 module.exports = [
4     { id: 1, nome: 'Alice', telefone: '1234-5678', email: 'alice@example.com' },
5     { id: 2, nome: 'Bob', telefone: '8765-4321', email: 'bob@example.com' },
6     { id: 3, nome: 'Carol', telefone: '5678-1234', email: 'carol@example.com' }
7 ];
```

Deletar usuário



```
1 //removerContato.js
2
3 let contatos = require('./contatos');
4
5 function removerContato(id) {
6   const index = contatos.findIndex(contato => contato.id === id);
7   if (index !== -1) {
8     contatos.splice(index, 1);
9   }
10 }
11
12 module.exports = removerContato;
```



```
1 //contatos.js
2
3 module.exports = [
4   { id: 1, nome: 'Alice', telefone: '1234-5678', email: 'alice@example.com' },
5   { id: 2, nome: 'Bob', telefone: '8765-4321', email: 'bob@example.com' },
6   { id: 3, nome: 'Carol', telefone: '5678-1234', email: 'carol@example.com' }
7 ];
```

O que é um ID?

IDs são como a "impressão digital" dos dados em um sistema. Eles servem para identificar de maneira única cada registro ou entidade dentro da aplicação.

Por que?

- **Unicidade**

Os ids garantem que cada registro seja único. Isso é crucial para que tenhamos certeza daquilo que estamos lidando com o dado correto.

Pra que?

- **Identificação**

Imagine tentar encontrar alguém em um país onde todos os homens se chamam João e todas as mulheres se chama Maria. Como vamos ter certeza se encontramos exatamente quem estamos buscando?

Por que?

- **Relacionamento**

Em sistemas, IDs são usados para criar relacionamento entre dados, facilitando a recuperação de informações.

Pra que?

- **Performance**

A busca por um ID é geralmente muito rápida, tendo em vista que os sistemas são otimizados para processá-los.

Exemplo prático

- Commit (Github)

```
→ contatos git:(master) x git commit -m "initial commit"
[master (root-commit) 1cdb8f2] initial commit
0 files changed, 101 insertions(+)
```


Como está sendo gerado o ID no exemplo demonstrado?

Menu de interação

```
1 // index.js
2 const prompt = require('prompt-sync')();
3 const listarContatos = require('./listarContatos');
4 const adicionarContato = require('./adicionarContato');
5 const atualizarContato = require('./atualizarContato');
6 const removerContato = require('./removerContato');
7
8 function mainMenu() {
9   console.log(`
10     1. Listar contatos
11     2. Adicionar contato
12     3. Atualizar contato
13     4. Remover contato
14     5. Sair
15   `);
16
17   const opcao = prompt('Escolha uma opção: ');
18
19   switch (opcao) {
20     case '1':
21       listarContatos();
22       mainMenu();
23     break;
```

```
24   case '2':
25     const nome = prompt('Nome: ');
26     const telefone = prompt('Telefone: ');
27     const email = prompt('Email: ');
28     adicionarContato({ id: Date.now(), nome, telefone, email });
29     console.log('Contato adicionado com sucesso!');
30     mainMenu();
31     break;
32   case '3':
33     const idAtualizar = parseInt(prompt('ID do contato a atualizar: '));
34     const novoNome = prompt('Novo nome: ');
35     const novoTelefone = prompt('Novo telefone: ');
36     const novoEmail = prompt('Novo email: ');
37     atualizarContato(idAtualizar, { nome: novoNome, telefone: novoTelefone, email: novoEmail });
38     console.log('Contato atualizado com sucesso!');
39     mainMenu();
40     break;
41   case '4':
42     const idRemover = parseInt(prompt('ID do contato a remover: '));
43     removerContato(idRemover);
44     console.log('Contato removido com sucesso!');
45     mainMenu();
46     break;
47   case '5':
48     break;
49   default:
50     console.log('Opção inválida!');
51     mainMenu();
52 }
53 }
54
55 mainMenu();
```

`Date.now()` == número de milisegundos decorrido
desde 01 de janeiro de 1970

Mudando a abordagem para gerar um id sequencial



```
1  //adicionarContato.js
2
3  let contatos = require('./contatos');
4
5  function adicionarContato(contato) {
6      contato.id = contatos.length + 1;
7      contatos.push(contato);
8  }
9
10 module.exports = adicionarContato;
```

Mudando a abordagem para gerar um id sequencial



```
1  case '2':  
2      const nome = prompt('Nome: ');  
3      const telefone = prompt('Telefone: ');  
4      const email = prompt('Email: ');  
5      adicionarContato({ nome, telefone, email });  
6      console.log('Contato adicionado com sucesso!');  
7      mainMenu();  
8      break;
```

Validando que o e-mail é único no sistema

Por que?

- **Integridade**

Garantir que cada e-mail seja único, mantén a integridade dos dados. Se você permitir duplicação, acabará com registros conflitantes e confusos.

Pra que?

- **Experiência do usuário**

Usuários podem se frustrar se puderem registrar múltiplas contas com o mesmo e-mail, causando confusão ao tentar lembrar qual conta foi realizado determinadas ações.

Por que?

- **Segurança**

Validação de e-mails únicos é uma medida de segurança. E-mails duplicados podem ser explorados por atacantes para tomar controle de múltiplas contas ou acessar dados sensíveis.

Pra que?

- **Comunicação eficiente**

Quando enviando e-mails para notificações de marketing, duplicatas resultam em mensagens redundantes para os mesmos usuários, aumentando os custos e reduzindo a eficácia da estratégia.

Implementando a validação de e-mail

```
1  //adicionarContato.js
2
3  let contatos = require("../contatos");
4
5  function adicionarContato(contato) {
6      contato.id = contatos.length + 1;
7
8      let existe = false;
9      for (let i = 0; i < contatos.length; i++) {
10         if (contatos[i].email === contato.email) {
11             existe = true;
12         }
13     }
14
15     if (!existe) {
16         contatos.push(contato);
17     }
18 }
19
20 module.exports = adicionarContato;
21
```


E como fica a experiência do usuário caso o email já esteja em uso?

Implementando a validação de e-mail

```
1 //adicionarContato.js
2
3 let contatos = require("./contatos");
4 let emailUsedByDifferentUser = require("./emailUsedByDifferentUser");
5
6 function adicionarContato(contato) {
7     contato.id = contatos.length + 1;
8
9     let isEmailInUse = emailUsedByDifferentUser(contato.email, contato.id);
10
11     if (!isEmailInUse) {
12         contatos.push(contato);
13         return true;
14     } else {
15         console.log("Email já cadastrado!");
16         return false;
17     }
18 }
19
20 module.exports = adicionarContato;
21
```

```
1 case '2':
2     const nome = prompt('Nome: ');
3     const telefone = prompt('Telefone: ');
4     const email = prompt('Email: ');
5     const resultadoAdicionar = adicionarContato({ nome, telefone, email });
6     if (resultadoAdicionar) {
7         console.log('Contato adicionado com sucesso!');
8     }
9     mainMenu();
10    break;
```

```
1 //emailAlreadyInUse.js
2 let contatos = require("./contatos");
3
4 function emailUsedByDifferentUser(email, id) {
5     for (let i = 0; i < contatos.length; i++) {
6         if (contatos[i].email === email && contatos[i].id !== id) {
7             return true;
8         }
9     }
10    return false
11 }
12
13 module.exports = emailUsedByDifferentUser;
14
```

Como ajustar o sistema para permitir vários telefones para o mesmo usuário?

Permitir vários números para o mesmo usuário

```
1 case '2':
2   const nome = prompt('Nome: ');
3
4   const email = prompt('Email: ');
5
6   let telefones = [];
7   let telefone;
8   while ((telefone = prompt('Telefone (deixe vazio para parar): '))) {
9     telefones.push(telefone);
10  }
11
12  const resultadoAdicionar = adicionarContato({ nome, telefones, email });
13  if (resultadoAdicionar) {
14    console.log('Contato adicionado com sucesso!');
15  }
16  mainMenu();
17  break;
```

```
1 case '3':
2   const idAtualizar = parseInt(prompt('ID do contato a atualizar: '));
3   const novoNome = prompt('Novo nome: ');
4
5   const novoEmail = prompt('Novo email: ');
6
7   let novosTelefones = [];
8   let novoTelefone;
9   while ((novoTelefone = prompt('Novo telefone (deixe vazio para parar): '))) {
10     novosTelefones.push(novoTelefone);
11   }
12
13   atualizarContato(idAtualizar, { nome: novoNome, telefones: novosTelefones, email: novoEmail });
14   console.log('Contato atualizado com sucesso!');
15   mainMenu();
16   break;
```

```
1 //contatos.js
2
3 module.exports = [
4   { id: 1, nome: 'Alice', telefones: ['1234-5678'], email: 'alice@example.com' },
5   { id: 2, nome: 'Bob', telefones: ['8765-4321'], email: 'bob@example.com' },
6   { id: 3, nome: 'Carol', telefones: ['5678-1234'], email: 'carol@example.com' }
7 ];
```

Requisitos obrigatórios

- Cadastro de usuário 
 - Deve ser gerado um id sequencial para cada usuário 
 - Deve ser adicionado o e-mail, nome e uma lista de telefones do usuário 
 - Não permitir o cadastro de usuário com e-mail duplicado 
 - Exibir mensagem de erro em caso de e-mail duplicado 

Requisitos obrigatórios

- Busca de usuários
 - O sistema deve listar todos os usuários com suas informações
 - Telefones devem ser exibidos como uma lista associada

Permitir vários números para o mesmo usuário



```
1  //listarContatos.js
2
3  let contatos = require('./contatos');
4
5  function listarContatos() {
6    for (let i = 0; i < contatos.length; i++) {
7      const contato = contatos[i];
8      console.log(`ID: ${contato.id}, Nome: ${contato.nome}, Email: ${contato.email}`);
9      console.log('Telefones:');
10     for (let j = 0; j < contato.telefones.length; j++) {
11       console.log(`  ${j + 1}. ${contato.telefones[j]}`);
12     }
13     console.log(''); // Linha em branco para separar contatos
14   }
15 }
16
17 module.exports = listarContatos;
18
```

Requisitos obrigatórios

- Busca de usuários 
 - O sistema deve listar todos os usuários com suas informações 
 - Telefones devem ser exibidos como uma lista associada 

Requisitos obrigatórios

- Atualização de usuário
 - O usuário pode atualizar o nome, o e-mail ou adicionar ou remover um telefone associado
 - Ao atualizar o e-mail, deve garantir o novo não está em uso
 - Exibir mensagem de erro se o novo e-mail já estiver em uso

Atualizando um usuário

```
1 //atualizarContato.js
2 const getIndexById = require("./getIndexById");
3 const emailUsedByDifferentUser = require("./emailUsedByDifferentUser");
4 let contatos = require("./contatos");
5
6 function atualizarContato(id, novoContato) {
7   const index = getIndexById(id)
8
9   if (index === -1) {
10     console.log("Contato não encontrado!");
11     return false;
12   }
13
14   if(emailUsedByDifferentUser(novoContato.email, id)){
15     console.log("Email já está em uso. Por favor, use um email diferente.");
16     return false
17   }
18
19   contatos[index].nome = novoContato.nome || contatos[index].nome;
20   contatos[index].email = novoContato.email || contatos[index].email;
21
22   if (novoContato.telefones.length > 0) {
23     contatos[index].telefones = novoContato.telefones;
24   }
25
26   return true;
27 }
28
29 module.exports = atualizarContato;
30
```

```
1 //getIndexById.js
2
3 let contatos = require("./contatos");
4
5 function getIndexById(id) {
6   const index = contatos.findIndex((contato) => contato.id === id);
7
8   if (index === -1) {
9     console.log("Contato não encontrado!");
10  }
11
12  return index;
13 }
14
15 module.exports = getIndexById
```

```
1 case '3':
2   const idAtualizar = parseInt(prompt('ID do contato a atualizar: '));
3   const novoNome = prompt('Novo nome (deixe vazio para não alterar: ');
4   const novoEmail = prompt('Novo email (deixe vazio para não alterar: ');
5
6   let novosTelefones = [];
7   let novoTelefone;
8   while ((novoTelefone = prompt('Novo telefone (deixe vazio para parar: '))) {
9     novosTelefones.push(novoTelefone);
10  }
11
12  const resultadoAtualizar = atualizarContato(idAtualizar, { nome: novoNome, email: novoEmail, telefones: novosTelefones });
13  if (resultadoAtualizar) {
14    console.log('Contato atualizado com sucesso!');
15  }
16
17  mainMenu();
18  break;
```

Requisitos obrigatórios

- Atualização de usuário 
 - O usuário pode atualizar o nome, o e-mail ou adicionar ou remover um telefone associado 
 - Ao atualizar o e-mail, deve garantir o novo não está em uso 
 - Exibir mensagem de erro se o novo e-mail já estiver em uso 

Requisitos obrigatórios

- Remoção de contato
 - Permitir a remoção de um contato especificando um ID
 - Solicitar mensagem de confirmação antes de proceder

Removendo um usuário

```
1 //removerContato.js
2 const getIndexById = require("../getIndexById");
3 let contatos = require("../contatos");
4
5 function removerContato(id, confirmacao) {
6     const index = getIndexById(id)
7
8     if (confirmacao.toLowerCase() !== "s") {
9         console.log("Desistindo de remover contato!");
10        return false;
11    }
12
13    if (index === -1) {
14        return false;
15    }
16
17    contatos.splice(index, 1);
18    return true;
19 }
20
21 module.exports = removerContato;
```

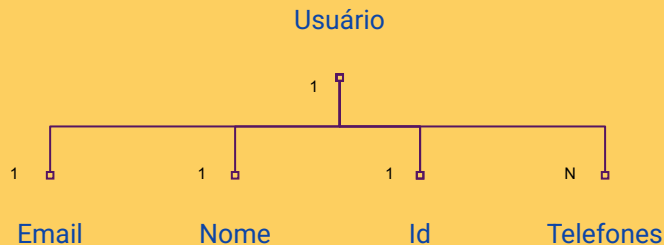
```
1 //getIndexById.js
2
3 let contatos = require("../contatos");
4
5 function getIndexById(id) {
6     const index = contatos.findIndex((contato) => contato.id === id);
7
8     if (index === -1) {
9         console.log("Contato não encontrado!");
10    }
11
12    return index;
13 }
14
15 module.exports = getIndexById;
```

```
1 case "4":
2     const idRemover = parseInt(prompt("ID do contato a remover: "));
3     const confirmacao = prompt(
4         "Tem certeza que deseja remover este contato? (s/n): "
5     );
6     const result = removerContato(idRemover, confirmacao);
7     if (result) {
8         console.log("Contato removido com sucesso!");
9     } else {
10        console.log("Contato não removido!");
11    }
12
13    mainMenu();
14    break;
```

Requisitos obrigatórios

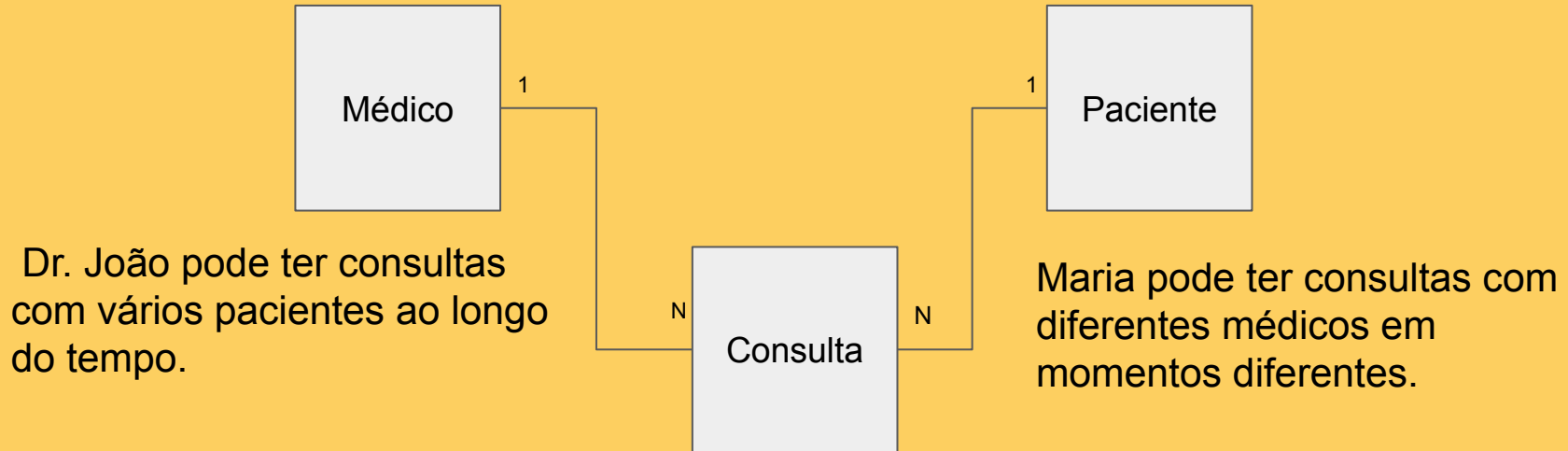
- Remoção de contato 
 - Permitir a remoção de um contato especificando um ID 
 - Solicitar mensagem de confirmação antes de proceder 

Relacionamento entre os dados



```
const usuario = {  
  id: 1,  
  nome: 'João Silva',  
  email: 'joao.silva@example.com',  
  telefones: ['123456789', '987654321']  
};
```

Relacionamento entre os dados



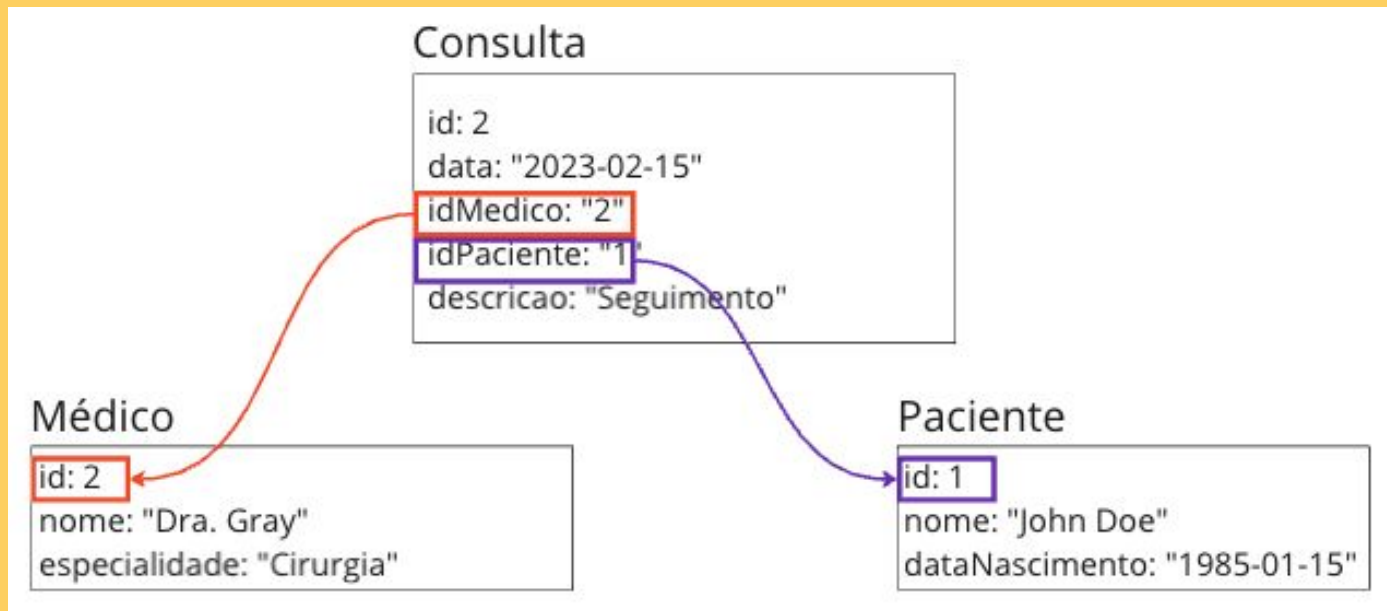
A consulta de Maria com Dr. João em 01/10/2024.

Relacionamento entre os dados

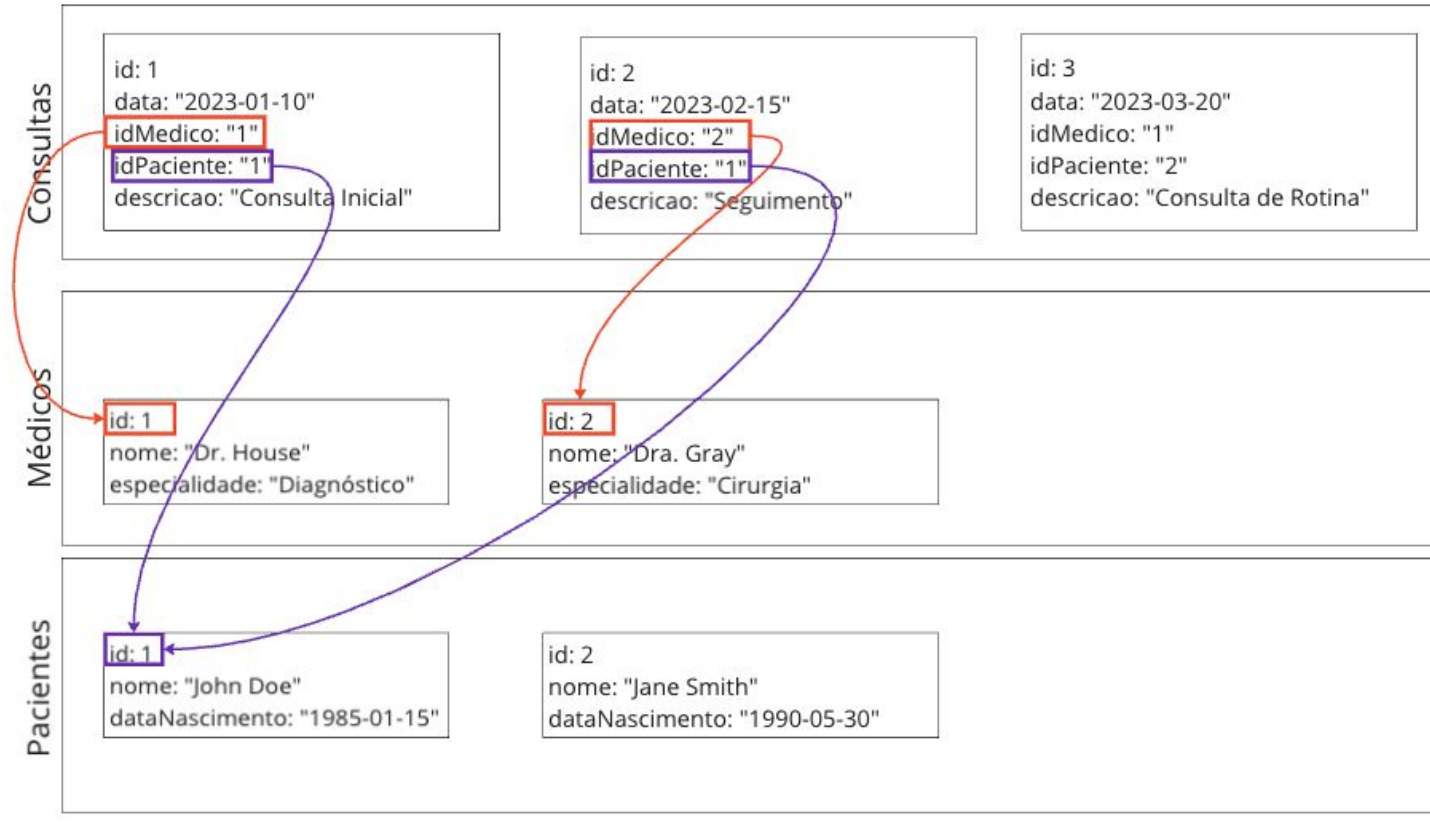


```
1  const medicos = [  
2    { id: 1, nome: 'Dr. House', especialidade: 'Diagnóstico' },  
3    { id: 2, nome: 'Dra. Grey', especialidade: 'Cirurgia' }  
4  ];  
5  
6  const pacientes = [  
7    { id: 1, nome: 'John Doe', dataNascimento: '1985-01-15' },  
8    { id: 2, nome: 'Jane Smith', dataNascimento: '1990-05-30' }  
9  ];  
10  
11  const consultas = [  
12    { id: 1, data: '2023-01-10', idMedico: 1, idPaciente: 1, descricao: 'Consulta inicial' },  
13    { id: 2, data: '2023-02-15', idMedico: 2, idPaciente: 1, descricao: 'Seguimento' },  
14    { id: 3, data: '2023-03-20', idMedico: 1, idPaciente: 2, descricao: 'Consulta de rotina' }  
15  ];  
16  
17  
18  module.exports = { medicos, pacientes, consultas };
```

Relacionamento entre os dados



Relacionamento entre os dados



Relacionamento entre os dados

```
1 //mostrarConsultas.js
2
3 const { consultas } = require("../dados");
4 const { encontrarMedicoPorId } = require("../encontrarMedicoPorId");
5
6 function mostrarConsultasPaciente(idPaciente) {
7   const consultasPaciente = consultas.filter(
8     (c) => c.idPaciente === idPaciente
9   );
10
11   consultasPaciente.forEach((consulta) => {
12     const medico = encontrarMedicoPorId(consulta.idMedico);
13
14     console.log(
15       ` - ID da Consulta: ${consulta.id}, Data: ${consulta.data}, Médico: ${medico.nome}, Descrição: ${consulta.descricao}`
16     );
17   });
18 }
19
20 mostrarConsultasPaciente(1);
21
```

```
1 //encontrarMedicoPorId.js
2
3 const { medicos } = require("../dados");
4
5 function encontrarMedicoPorId(idMedico) {
6   for (let i = 0; i < medicos.length; i++) {
7     if (medicos[i].id === idMedico) {
8       return medicos[i];
9     }
10   }
11 }
12
13 module.exports = { encontrarMedicoPorId };
14
```

Relacionamento entre os dados

- Executando passando o idPaciente === 1 como parametro

Filtragem das consultas

```
const consultasPaciente = consultas.filter(  
  (c) => c.idPaciente === idPaciente  
);
```

Iteração sobre as consultas filtradas

```
consultasPaciente.forEach((consulta) => {
```

Relacionamento entre os dados

- Primeira iteração

Encontrando o médico responsável pela primeira consulta

```
12     const medico = encontrarMedicoPorId(consulta.idMedico);  
13
```

Imprimindo detalhes da primeira consulta

```
    console.log(  
      ` - ID da Consulta: ${consulta.id}, Data: ${consulta.data}, Médico: ${medico.nome}, Descrição: ${consulta.descricao}`  
    );  
  });
```

Relacionamento entre os dados

- Segunda iteração

Encontrando o médico responsável pela primeira consulta

```
12     const medico = encontrarMedicoPorId(consulta.idMedico);  
13
```

Imprimindo detalhes da primeira consulta

```
    console.log(  
      ` - ID da Consulta: ${consulta.id}, Data: ${consulta.data}, Médico: ${medico.nome}, Descrição: ${consulta.descricao}`  
    );  
  });
```

Relacionamento entre os dados

- Final da execução

```
- ID da Consulta: 1, Data: 2023-01-10, Médico: Dr. House, Descrição: Consulta inicial  
- ID da Consulta: 2, Data: 2023-02-15, Médico: Dra. Grey, Descrição: Seguimento
```